

DesignDojo: an LLD practice platform

Practise Low-Level Design the way you practise algorithms: pick a problem, submit a design, get feedback that points at your actual classes, and try again with the last review still in view.

Built for the CipherSchools 2-day engineering assignment.

Research note	docs/RESEARCH.md : the learner problem, what exists, the gaps, and the product direction
Design note	docs/DESIGN.md : MVP, flow, classes, evaluation approach, trade-offs
AI usage	AI_USAGE.md : five decisions where AI changed the outcome, including the ones I rejected

Quick start

Requires **Node 20+**. No database and no API key needed.

```
npm install
npm run dev
```

- App → <http://localhost:5173>
- API → <http://localhost:4000/api/health>

That is the whole setup. With no `MONGODB_URI` the platform stores attempts in memory, and with no `ANTHROPIC_API_KEY` it uses an offline stand-in reviewer, labelled as such in the UI, so the full loop works on a fresh clone.

With the real reviewer model

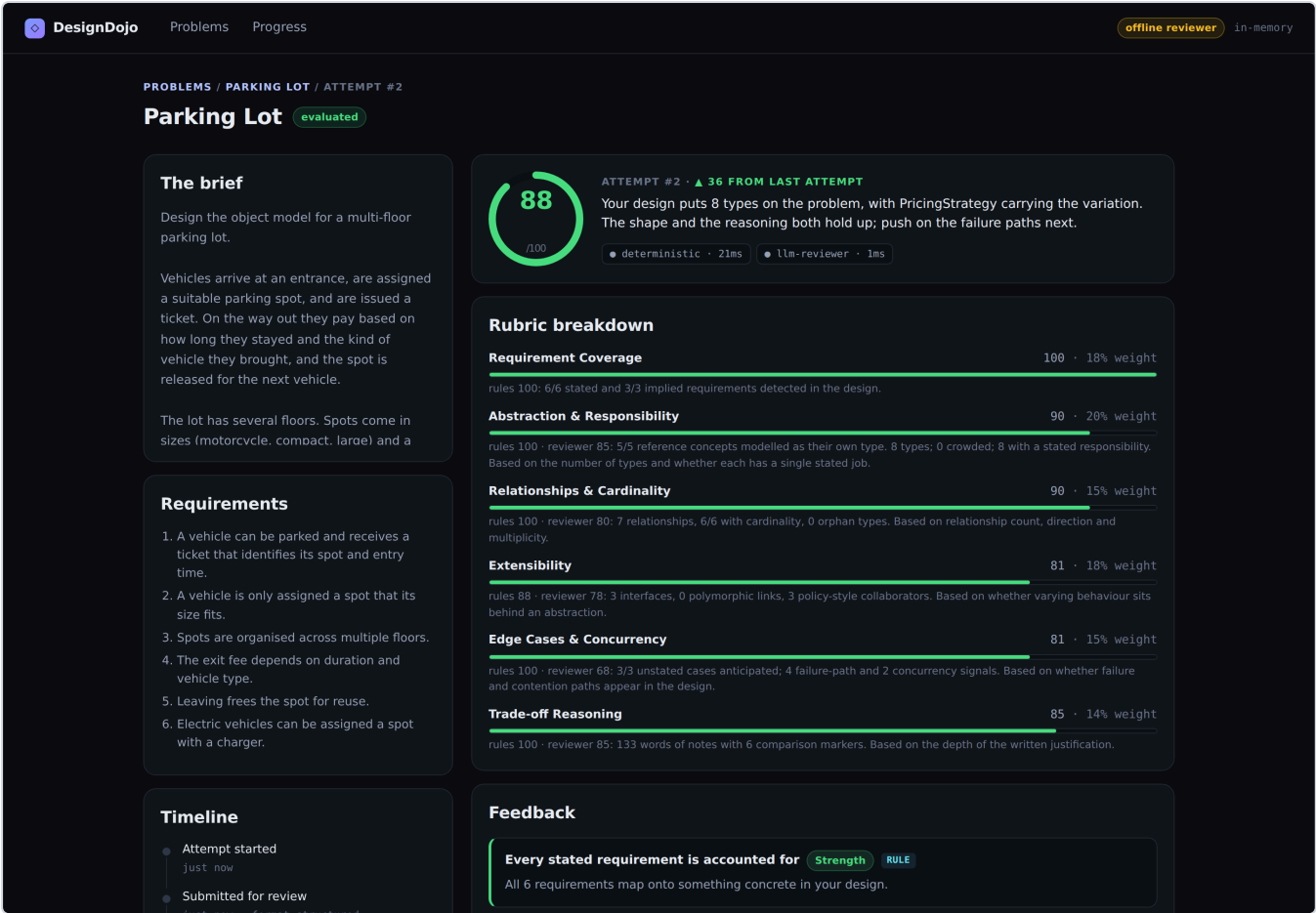
```
cp .env.example .env
# add ANTHROPIC_API_KEY=sk-ant-...
npm run dev
```

The header badge switches from `offline reviewer` to the model id. To persist attempts, add `MONGODB_URI=mongodb://localhost:27017/lld-practice`; the badge then reads `mongodb`.

Tests

```
npm test      # 85 tests
npm run typecheck
```

The loop



Choose → design → submit → review → try again.

DesignDojo

ProblemsProgress

offline feedback!in memory

PRACTICE

Pick a problem

Design it, submit it, and get feedback that points at your actual classes. Every attempt is kept, so you can see what changed when you try again.

Parking Lot

easy

40 min

best 88

2 attempts

Vending Machine

easy

30 min

best 37

1 attempt

Elevator System

hard

30 min

6 requirements

not attempted

Expense Splitter

medium

40 min

3 requirements

not attempted

Catalogue: four problems, your best score on each

DesignDojo

ProblemsProgress

offline feedback!in memory

PROBLEMS / ELEVATOR SYSTEM / ATTEMPT #1

Elevator System

draft saved just now

Submit for review

Submit for review

The brief

Design the object model for the elevator system of an office building.

The building has several floors, and several elevator cars. A person on a floor presses up or down, a person inside a car presses a destination floor. The system decides which car serves which request and in what order, then the cars move, stop and open their doors.

Cars can be taken out of service for

Requirements

1. A person on a floor can request a car by direction.

2. A person inside a car can request a destination floor.

3. The system chooses which car serves each request.

4. A car moves between floors, stops, and opens its doors.

5. A car serves its pending stops in a sensible order rather than strictly first-come.

6. A car can be out of service or restricted to certain floors.

Timeline

Attempt started

just now

Structured design

Code

Class diagram

Classes & interfaces

One line per type. The responsibility is what gets reviewed, if it needs an "and", you probably have two classes.

Parkinglot

class

Aggregate root that owns the floors and coordinates park(vehicle), unpark(ticket)

Parkingfloor

class

Holds the spots on one level and reports which are free(available(size)

ParkingSpot

class

A single allocatable space with a size and current size, currentTicket, hasCharger

Vehicle

class

The vehicle being parked, knowing its plate and si licensePlate, size, isElectric

Relationships

Why does a reference to whom, and how many. Cardinality is graded.

Parkinglot

composition

Parkingfloor

1..*

Parkingfloor

composition

ParkingSpot

1..*

Workspace: the brief beside the design, autosaved

DesignDojo

ProblemsProgress

offline feedback!in memory

PROBLEMS / PARKING LOT

Parking Lot

easy

40 min

best 88/100

The problem

Design the object model for a multi-floor parking lot.

Vehicles arrive at an entrance, are assigned a suitable parking spot, and are issued a ticket. On the way out they pay based on how long they stayed and the kind of vehicle they brought, and the spot is released for the next vehicle.

The lot has several floors. Spots come in sizes (motorcycle, compact, large) and a vehicle may only occupy a spot that fits it. Some spots are reserved for electric vehicles and have a charger attached.

Model the classes, their responsibilities, and the relationships between them. You are not being asked for a working implementation, a database schema, or an API, just the design.

Requirements

1. A vehicle can be parked and receives a ticket that identifies its spot and entry time.

2. A vehicle is only assigned a spot that fits its size.

3. Spots are organized across multiple floors.

4. The exit fee depends on duration and vehicle type.

5. Leaving frees the spot for reuse.

6. Electric vehicles can be assigned a spot with a charger.

Constraints

A single physical lot: you do not need to model a chain of lots.

Payment is taken at exit: You do not need to integrate a real payment provider: model the boundary.

Assume up to a few thousand spots, so an in-memory index of free spots is reasonable.

Questions your design should be able to answer

Try again

You will be asked to reuse your classes, their responsibilities and how they relate, or you can submit lots or a full model class diagram instead.

How this is scored

Requirement Coverage18%

Abstraction & Responsibility20%

Relationships & Cardinality15%

Extensibility19%

Edge Cases & Concurrency15%

Trade-off Reasoning14%

Your attempts (2)

#288/100just now

#152/100just now

Brief: requirements, constraints, and how it is scored

DesignDojo

ProblemsProgress

offline feedback!in memory

PROGRESS

Your attempts

3 attempts across 2 problems - average score 58

Vending Machine

1 attempt

#137/100

weakest: Trade-off Reasoning 15

just now - structured

Parking Lot

2 attempts - improved 36 points

#288/100

weakest: Extensibility 81

just now - structured

#152/100

weakest: Trade-off Reasoning 15

just now - structured

Progress: every attempt kept and charted

What makes the feedback useful

There is more than one correct design for a parking lot, so grading against a reference solution would be actively wrong. Three things fall out of taking that seriously.

1. Evaluation is split between rules and a model

Checkable things are decided by reproducible rules; judgement calls go to a language model. The rubric declares the blend **per dimension**:

Dimension	Rules	Model	Why
Requirement Coverage	70%	30%	A finite authored checklist: mostly a fact
Relationships & Cardinality	50%	50%	Presence is a fact; correctness is judgement
Edge Cases & Concurrency	40%	60%	Vocabulary is detectable; sufficiency is not
Abstraction & Responsibility	30%	70%	Crowded classes are detectable; good taste is not
Extensibility	30%	70%	Seams are detectable; the <i>right</i> seams are not
Trade-off Reasoning	0%	100%	Only a reader can judge an argument

The deterministic half is the platform's floor: dependency-free, single-digit milliseconds, identical every run. The model is given the rule findings and told not to repeat them, so the two compose into one review.

2. Every finding says where it came from

Each item is tagged `rule` or `reviewer` on screen. Requirement coverage shows the learner's own words that matched. A verdict you disagree with is something you can argue with, not something handed down, which matters a lot when the model is occasionally wrong.

3. Rules ask rather than assert

No check says a concept is missing. It says nothing was *detected* that owns it, and asks whether that was deliberate, with a score floor, so departing from the reference costs some marks and never all of them.

Concepts are graded three ways rather than two: **dedicated** (a type is named for it), **folded** (it exists only as a method on another class, half credit and a question), or **absent**. Full credit for folding it in would let name-dropping score as well as modelling; no credit would assert that one decomposition is the only right one.

When evaluation is slow or fails

Evaluation is asynchronous, so submitting returns immediately with the attempt `queued` and the client polls. Three levels of failure handling, and at none of them is the learner's work lost:

1. **The model fails or returns unparseable JSON** → the review completes from the deterministic checks alone, flagged `degraded` with the reason, and the UI shows a partial-review banner with a re-run button.
 2. **The whole run fails retryably** → re-queued with exponential backoff (1s, 2s, 4s), up to 3 runs.
 3. **Runs exhausted** → the attempt lands in `failed` with the reason kept and the submission intact; the learner re-runs it with a fresh budget.
-

Architecture

web/	React + Vite, no UI framework
server/src/	
domain/	← no imports from anything below this line
problem/	Problem, Rubric (dimensions + deterministic share)
attempt/	Attempt aggregate, state machine, Submission union, DesignModel
evaluation/	Evaluation value objects
evaluation/	
normalizers/	Structured Code Diagram → DesignModel (Strategy)
deterministic/checks/	8 independent DesignChecks
llm/	LlmClient (Anthropic offline), prompt, schema-validated parse
EvaluationPipeline.ts	composes evaluators, degrades gracefully
ScoreAggregator.ts	blends by the rubric's deterministic share
application/	AttemptService, EvaluationWorker, JobQueue port
infrastructure/	repositories (in-memory Mongo), HTTP, seeded problems
container.ts	the only file that names a concrete adapter

Dependencies point inward. The domain imports nothing from `infrastructure`, which is what lets the whole suite run against in-memory adapters in about a second.

Extending it

To add...	You write	You do not touch
A submission format	one <code>SubmissionNormalizer</code>	any evaluator, service or route
An evaluation approach	one <code>Evaluator</code>	the pipeline, worker or lifecycle
A problem	one entry in <code>seedProblems()</code>	no code at all
A real queue / DB / model provider	one adapter	one line in <code>container.ts</code>

API

All routes are under `/api`. The learner is identified by an `x-learner-id` header (see *Limitations*).

Method	Route	Purpose
GET	<code>/health</code>	Storage and reviewer actually in use, queue depth
GET	<code>/problems</code>	Catalogue
GET	<code>/problems/:idOrSlug</code>	Brief, requirements, constraints, rubric weights
POST	<code>/attempts</code>	Start an attempt ({ problemId })
PUT	<code>/attempts/:id/draft</code>	Autosave
POST	<code>/attempts/:id/submit</code>	Submit; returns immediately as <code>queued</code>
GET	<code>/attempts/:id</code>	Poll for status and the evaluation
POST	<code>/attempts/:id/reevaluate</code>	Re-run a failed evaluation
GET	<code>/attempts?problemId=</code>	History

The grading key (requirement signals, reference concepts, pitfalls) never leaves the server; there is a test asserting it.

Tests

85 tests, ~1.5s, no external services.

File	Covers
<code>attempt-lifecycle.test.ts</code>	State machine: happy path, illegal transitions, retry budget, and that a rejected submission leaves the attempt editable
<code>evaluation-pipeline.test.ts</code>	Strong vs weak scoring separation, that the weak design is blamed for the <i>right</i> dimensions, reproducibility, the rule/model blend, and four degradation paths (model down, prose instead of JSON, wrong schema, fenced JSON)
<code>evaluation-worker.test.ts</code>	Retry with backoff, exhausting the budget into <code>failed</code> , learner-triggered re-run, redelivered jobs, previous-score context
<code>normalizers.test.ts</code>	TypeScript / Java / Python parsing, Mermaid arrow direction and cardinality, unparseable input, unregistered format
<code>scoring.test.ts</code>	Rubric normalisation and validation, blending, renormalising over assessed dimensions, clamping, JSON extraction edge cases
<code>api.test.ts</code>	End-to-end loop, repeat attempts showing improvement, cross-learner isolation, 422/409/404 paths, running with no reviewer at all
<code>problem-catalogue.test.ts</code>	Every seeded problem is completely authored, its rubric covers all six dimensions, its signals are actually matchable, and it separates a strong design from a generic one: a guard that covers each problem added later with no new fixture
<code>signal-matching.test.ts</code>	Word-boundary and inflection rules, camelCase/snake_case identifier splitting, and that a dedicated type scores above the same concept folded into a god class

Limitations

Honest about what this MVP is not:

- **No accounts.** A per-browser id in `localStorage`; clearing it loses your history. Isolated to one function on each side.
- **In-process queue.** A restart loses queued jobs, leaving the attempt in `queued` for a re-run. The `JobQueue` interface is the upgrade path.
- **Requirement coverage is signal matching, not comprehension.** It can be gamed by name-dropping and can miss unusual vocabulary. It shows its evidence precisely so you can see when it is wrong.
- **Code parsing is regex-based**, not an AST. It reads shape, not semantics; the model reads the raw source anyway.
- **The offline reviewer is not a model.** It is a heuristic stand-in so the loop works without a key, and it says so in the UI and in the evaluation's provenance.

- **Four problems**, seeded in code. Adding more is a content task, not an engineering one.